

CONCETTI DI ARCHITETTURA ED ORGANIZZAZIONE

ARCHITETTURA DI UN CALCOLATORE

- RIGUARDA CIÒ CHE È VISIBILE AL PROGRAMMATORE
- COMPRENDE TUTTI GLI ASPETTI CHE INFLUENZANO DIRETTAMENTE LA SCRITTURA DEL SOFTWARE, COME: IL SET DI ISTRUZIONI, LE DIMENSIONI DEI REGISTRI, LA MODALITÀ DI INDIRIZZAMENTO.
- È ORIENTATA ALLA PROGRAMMAZIONE E RAPPRESENTA L'INTERFACCIA TRA HARDWARE E SOFTWARE
- ES: UN'ISTRUZIONE MUL.F CHE PERMETTE DI FARE MOLTIPLICAZIONI IN VIRGOLA MOBILE FA PARTE DELL'ARCHITETTURA, PERCHÉ IL PROGRAMMATORE LA PUÒ USARE NEL CODICE

ORGANIZZAZIONE

- DESCRIVE COME L'ARCHITETTURA È EFFETTIVAMENTE REALIZZATA A LIVELLO FISICO, QUINDI RIGUARDA: LE UNITÀ FUNZIONALI (ALU, REGISTRI, BUS, CACHE,...), LE INTERCONNESSIONI TRA LE VARIE COMPONENTI
- È ORIENTATA ALL'HARDWARE, CIÒ ALLA REALIZZAZIONE CONCRETA
- ES: COME VIENE FISICAMENTE REALIZZATA L'ISTRUZIONE MUL.F È UN ASPETTO DELL'ORGANIZZAZIONE

TUTTI I PROCESSORI INTEL X86 AVEVANO LA STESSA ARCHITETTURA, MA ORGANIZZAZIONI DIVERSE. MANTENERE UGUALE L'ARCHITETTURA È IMPORTANTE PER LA PORTABILITÀ DEL SOFTWARE: LO STESSO PROGRAMMA GIÀ SU PIÙ MACCHINE DIVERSE. TUTTAVIA, SE SI CAMBIA TROPPO L'ORGANIZZAZIONE, A VOCE DI DIVENTARE NECESSARIO CAMBIARE ANCHE L'ARCHITETTURA

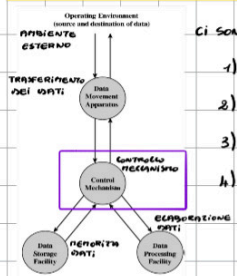
CONCETTI DI STRUTTURA E FUNZIONE

UN CALCOLATORE ELETTRONICO È UN SISTEMA COMPLESSO, COMPOSTO DA MODULI E SOTTOMODULI (LIVELLI DI ASTRAZIONE). PER CAPIRE IL FUNZIONAMENTO DEL SISTEMA SI USANO LIVELLI DI ASTRAZIONE: AD OGNI LIVELLO SI ANALIZZANO

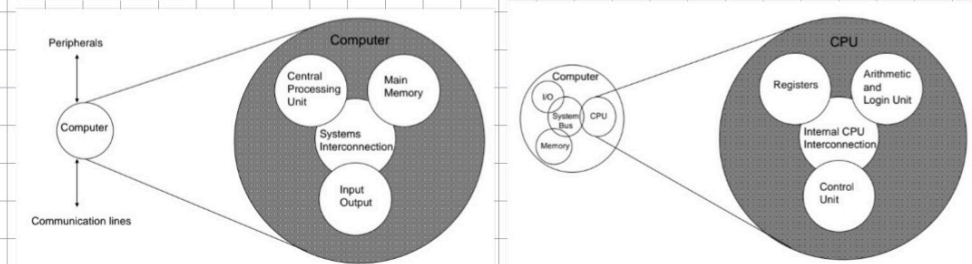
- STRUTTURA: COME I MODULI SONO INTERCONNESSI TRA LORO
- FUNZIONE: COSA FA CIASCUN MODULO, CIÒ IL SUO COMPORTAMENTO FUNZIONALE ALL'INTERNO DEL SISTEMA

ESEMPIO:
STRUTTURA → LA CPU È COLLEGATA ALLA MEMORIA RAM TRAMITE UN BUS DATI
FUNZIONE → LA CPU ESEGUE ISTRUZIONI, LA RAM MEMORIZZA DATI TEMPORANEI

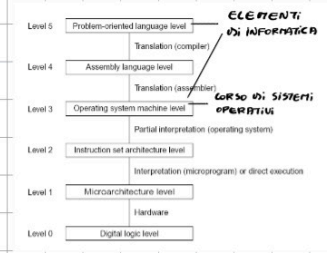
FUNZIONE DI UN CALCOLATORE



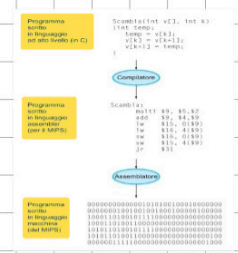
- CI SONO 4 FUNZIONI FONDAMENTALI CHE OGNI CALCOLATORE ELETTRONICO DEVE SVOLGERE
- 1) ELABORAZIONE DATI → LA CPU ESEGUE OPERAZIONI LOGICHE ED ARITMETICHE SUI DATI
 - 2) MEMORIZZAZIONE DATI → I DATI DEVONO ESSERE MEMORIZZATI TEMPORANEAMENTE (RAM) O PERMANENTEMENTE (DISCO, ROM,...)
 - 3) TRASFERIMENTO DATI → I DATI DEVONO POTER FLUIRE TRA I MODULI INTERNI E VERSO L'ESTERNO (INPUT/OUTPUT)
 - 4) CONTROLLO → UN MECCANISMO (GENERALMENTE LA CPU) COORDINA TUTTE LE OPERAZIONI ED ASSICURA CHE AVENGANO NEL GIUSTO ORDINE



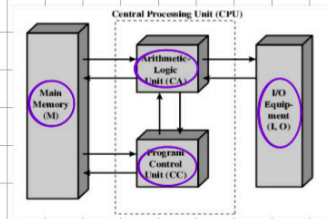
CALCOLATORE COME MACCHINA "VIRTUALE" - LIVELLI DI ASTRAZIONE



- OGNI CALCOLATORE PUÒ ESSERE VISTO COME COMPOSTO DA PIÙ LIVELLI, DOVE OGNI LIVELLO SIMULA O INTERPRETA IL LIVELLO INFERIORE
- LIVELLO 5 - LINGUAGGIO DI ALTO LIVELLO (C, JAVA, PYTHON,...). PROGRAMMI SCRITTI PER RISOLVERE PROBLEMI REALI
 - LIVELLO 4 - LINGUAGGIO SIMBOLICO SPECIFICO PER L'ARCHITETTURA (ES. MIPS, X86)
 - LIVELLO 3 - INTERFACCIA E SERVIZI DEL SISTEMA OPERATIVO (GESTIONE FILE, MEMORIA, PROCESSI,...)
 - LIVELLO 2 - SET DI ISTRUZIONI MACCHINA CHE LA CPU PUÒ ESEGUIRE. INTERFACCIA HARDWARE/SOFTWARE
 - LIVELLO 1 - COME L'ISA È IMPLEMENTATA FISICAMENTE (ALU, REGISTRI, BUS,...)
 - LIVELLO 0 - LIVELLO BASE (CIRCUITI LOGICI, PORTE, FLIP-FLOP,...)



ARCHITETTURA DI VON NEUMANN

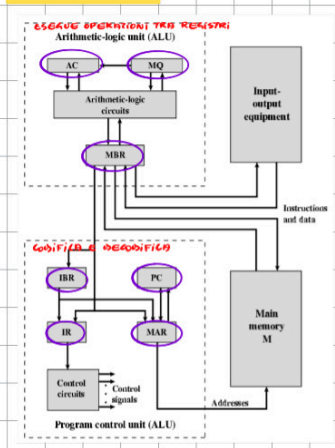


- IL MODELLO DI VON NEUMANN È IL MODELLO TEORICO DI BASE PER LA MAGGIOR PARTE DEI CALCOLATORI OGGI, PROPOSTO DA JOHN VON NEUMANN NEL 1945, ED È COMPOSTO:
- MAIN MEMORY (M) ⇒ MEMORIZZA ISTRUZIONI E DATI (NELLA STESSA MEMORIA ⇒ PRINCIPIO DI MEMORIZZAZIONE UNIFICATA)
 - CENTRAL PROCESSING UNIT (CPU) ⇒ ARITHMETIC-LOGIC UNIT (ALU) ⇒ ESEGUE OPERAZIONI ARITMETICHE E LOGICHE
 - PROGRAM CONTROL UNIT (PCU) ⇒ GESTISCE IL FLUSSO DELLE ISTRUZIONI, INTERPRETA ED ESEGUE I COMANDI
 - I/O EQUIPMENT (I, O) ⇒ DISPOSITIVI DI INPUT/OUTPUT PER COMUNICARE

CARATTERISTICHE FONDAMENTALI

- MEMORIA CONDIVISA PER ISTRUZIONI E DATI
- ESECUZIONE SEQUENZIALE DELLE ISTRUZIONI (UNA ALLA VOLTA)
- USA IL PROGRAM COUNTER PER TENERE TRACCIA DELL'ISTRUZIONE CORRENTE
- ARCHITETTURA UNIVERSALE: QUALUNQUE ALGORITMO PUÒ ESSERE ESEGUITO DA QUESTA MACCHINA

CALCOLATORE IAS



È LA REALIZZAZIONE CONCRETA DEL MODELLO DI VON NEUMANN, SVILUPPATA NEL 1952 AL PRINCETON INSTITUTE FOR ADVANCED STUDY (IAS)

MEMORIA → 1000 PAROLE DA 40 BIT. OGNI PAROLA PUÒ CONTENERE 2 ISTRUZIONI DA 20 BIT (UNA A SINISTRA, UNA A DESTRA)

REGISTRI PRINCIPALI:

- MBR (MEMORY BUFFER REGISTER) → CONTIENE IL DATO LETTO/SCRITTO DALLA MEMORIA
- MAR (MEMORY ADDRESS REGISTER) → CONTIENE L'INDIRIZZO DELLA CELLA DA LEGGERE/SCRIVERE
- IR (INSTRUCTION REGISTER) → CONTIENE L'ISTRUZIONE IN ESECUZIONE
- IBR (INSTRUCTION BUFFER REGISTER) → CONTIENE LA SECONDA ISTRUZIONE DA SEGUIRE
- PC (PROGRAM COUNTER) → CONTIENE L'INDIRIZZO DELLA PROSSIMA ISTRUZIONE
- AC (ACCUMULATOR) → REGISTRO PER RISULTATI INTERMEDI
- MQ (MULTIPLIER QUOTIENT) → USATO PER MOLTIPLICAZIONI E DIVISIONI

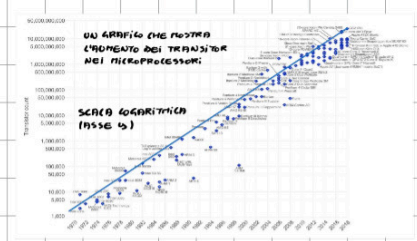
1. Il Program Counter (PC) indica l'indirizzo dell'istruzione da prelevare.
2. L'istruzione viene caricata nel MBR, poi spezzata tra IR e IBR.
3. Viene eseguita una parte, e la seconda memorizzata per dopo.
4. La Control Unit genera i segnali di controllo per attivare l'ALU o accedere alla memoria.
5. I risultati delle operazioni vengono salvati in AC o MQ.

LEGGE DI MOORE

FORMULATA DA GORDON MOORE (CO-FONDATORE DI INTEL) NEL 1965 → IL NUMERO DI TRANSISTOR IN UN CIRCUITO INTEGRATO RADDOPPIA OGNI ANNO (IN SEGUITO RIVISTA A RADDOPPIO OGNI 18 MESI)

EFFETTI OSSERVATI (ANNI '70-'10). FINO AGLI ANNI 2010 LA LEGGE SI È DIMOSTRATA SORPRENDENTEMENTE ACCURATA. AUMENTARE I TRANSISTOR HA COMPORTATO PRESTAZIONI SUPERIORI, RIDUZIONE DEI CONSUMI, MAGGIORE AFFIDABILITÀ

● NON È UNA LEGGE SCIENTIFICA FORMALE, MA UNA REGOLA EMPIRICA, BASATA SU OSSERVAZIONE STORICA DEL PROGRESSO



LA FINE DELLA LEGGE DI MOORE

→ VISIONE PESSIMISTA (ES. NVIDIA)

La legge è "morta e sepolta": Incrementare i transistor aumenta i costi in modo insostenibile. Più potenza → più calore e consumo, che va contro le esigenze moderne (mobile, efficienza). Si preferisce ottimizzare il software invece di puntare solo sull'hardware

→ VISIONE OTTIMISTA (ES. INTEL)

La legge è ancora "viva e vegeta": Nuovi materiali (es. grafene, GaN) possono rivoluzionare l'integrazione. Il quantum computing potrebbe permettere salti ancora maggiori. Continuerà a guidare l'evoluzione tecnologica almeno per un decennio.